

Programación concurrente

La programación concurrente es un paradigma de programación en el que varios procesos o hilos de ejecución pueden ejecutarse de manera simultánea.

No necesariamente estos procesos se ejecutan en términos temporales a la vez (salvo que existan múltiples núcleos de CPU), pero sí pueden compartir el tiempo y ejecutarse de forma simultánea; avanzan sin bloquearse entre sí.

Ejemplo: Contando números de forma concurrente

Crearemos un programa donde dos hilos cuentan números, pero no de manera secuencial, sino que ambos hilos trabajan en paralelo.

Código en Python:

```
import threading
import time
```

Función que va a ejecutar cada hilo:

```
def contar_numeros(nombre, contador):
    for i in range(1, 6):
        time.sleep(1)  # Simula un trabajo que
toma tiempo (1 segundo por número)
        print(f"{nombre} está contando: {contador
+ i}")
```

Creamos dos hilos:

```
hilo1 = threading.Thread(target=contar_numeros,
args=("Hilo 1", 0))
hilo2 = threading.Thread(target=contar_numeros,
args=("Hilo 2", 5))
```

Iniciamos ambos hilos:

```
hilo1.start()
hilo2.start()
```

Esperamos a que ambos hilos terminen:

```
hilo1.join()
hilo2.join()
print("¡Contador completo!")
```

SALIDA:

```
Hilo 1 está contando: 1
Hilo 2 está contando: 5
Hilo 1 está contando: 2
Hilo 2 está contando: 6
Hilo 1 está contando: 3
Hilo 2 está contando: 7
Hilo 1 está contando: 4
Hilo 2 está contando: 8
Hilo 1 está contando: 5
Hilo 2 está contando: 9

¡Contador completo!
```

¿Qué sucede en este ejemplo?

Ambos hilos se ejecutan de manera concurrente, lo que significa que el tiempo de ejecución de cada hilo se solapa.

Aunque cada hilo tiene una pausa de 1 segundo, como ambos están trabajando al mismo tiempo, sus salidas se intercalan.

Los números que imprimen los hilos no siguen un orden secuencial estricto debido a la concurrencia, y dependen del planificador del sistema operativo, que decide cuándo cada hilo obtiene tiempo de CPU.



¿Qué es un hilo?

Se entiende por hilo (en inglés, thread) a la unidad de ejecución más sencilla de un proceso. En la programación concurrente, los hilos permiten la ejecución de un conjunto de tareas de forma simultánea dentro de un mismo proceso (que comparte algunos de los recursos, como es el caso de la memoria), si bien funcionan de forma independiente y concurrente entre ellos.

¿Cuáles son las diferencias entre hilo y proceso?

▼ Proceso

Un proceso es una instancia activa de un programa. Cada proceso cuenta con su propio espacio de memoria y recursos asociados.

▼ Hilo

Un hilo es un componente que reside dentro de un proceso.

Todos los hilos en un mismo proceso comparten el espacio de memoria y recursos (tales como variables globales y archivos abiertos), pero cada uno tiene su propio contador de programa, pila y registros, lo que les permite funcionar de forma independiente.

Características de los hilos



1 Concurrencia

Los hilos permiten que las tareas dentro de un proceso se ejecuten "al mismo tiempo" (concurrentemente). Esto no necesariamente significa que se ejecutan al mismo tiempo en un procesador físico (a menos que tengas múltiples núcleos de CPU), pero sí significa que se alternan para realizar su trabajo de forma eficiente.

2 Independencia

Cada hilo puede ejecutar su propio código, pero al ser parte de un proceso común, puede acceder y modificar datos compartidos (aunque esto requiere cuidado para evitar errores de sincronización).

3 Ligereza

Los hilos son más ligeros que los procesos porque comparten el mismo espacio de memoria, lo que hace que crear y gestionar hilos sea más eficiente en términos de recursos que crear nuevos procesos.

Ejercicios para la clase 1



Problema 1: contando en paralelo

Escribe un programa en Python que cree dos hilos. El primer hilo debe contar los números del 1 al 5, mientras que el segundo hilo debe contar los números del 6 al 10. Los números de cada hilo deben imprimirse de forma concurrente, pero no necesariamente en el mismo orden.

Sincronización de hilos

Crea un programa que simule dos hilos sumando números. Ambos hilos deben sumar números del 1 al 5, pero no deben imprimir el resultado de la suma hasta que ambos hayan completado la suma de los 5 números. Utiliza una **variable de condición** para asegurar que los resultados solo se impriman una vez que ambos hilos hayan terminado su tarea.